MODUL 7 · DATA FETCHING & CACHING

Fetch Data di Server Components



Contoh yang sama dengan video sebelumnya, tapi di sisi server

Server Component Boleh async

Komponennya langsung menunggu datanya. Tidak ada hook sama sekali.

Yang tidak dipakai lagi

Dibanding Client Component

- useEffect
- useState untuk data, loading, dan error
- AbortController untuk membatalkan request

Yang dipakai

Cukup JavaScript biasa

- async function sebagai komponen
- await langsung di dalam komponen
- Tanpa 'use client'

Menyalin dari Versi Client

Endpoint yang sama, /api/produk. Tidak ada 'use client', komponennya async.

app/produk/page.tsx

```
type Produk = { id: number; title: string }

export default async function Page() {
  const res = await fetch('/api/produk')
  const produk: Produk[] = await res.json()

  return (
    <ul>
      {produk.map((p) => (
        <li key={p.id}>{p.title}</li>
      ))}
    </ul>
  )
}
```

Hasilnya: Langsung Gagal

Pesan ini muncul saat saya menjalankan next build di Next.js 16.

Terminal

```
// TypeError: Failed to parse URL from /api/produk
//   [cause]: TypeError: Invalid URL
//     code: 'ERR_INVALID_URL'
//     input: '/api/produk'
```

Kenapa gagal?

Di browser, '/api/produk' otomatis dilengkapi dengan domain halaman. Di server tidak ada halaman sebagai acuan, jadi alamat harus lengkap, misalnya http://localhost:3000/api/produk.

Pakai URL Lengkap

Satu baris yang berubah. Ini jalan selama API-nya hidup.

app/produk/page.tsx

```
type Produk = { id: number; title: string }

export default async function Page() {
  const res = await fetch('http://localhost:3000/api/produk')
  const produk: Produk[] = await res.json()

  return (
    <ul>
      {produk.map((p) => (
        <li key={p.id}>{p.title}</li>
      ))}
    </ul>
  )
}
```

Contoh Function Pengambil Data

Di Client Component, API route wajib ada karena browser tidak boleh menyentuh database. Di server, tidak perlu.

lib/produk.ts

```
export type Produk = { id: number; title: string }

export async function getProduk(): Promise<Produk[]> {
  // di proyek nyata: query Prisma ke database
  return [
    { id: 1, title: 'Kopi' },
    { id: 2, title: 'Teh' },
    { id: 3, title: 'Roti' },
  ]
}
```

Page Memanggil Function Langsung

Tanpa fetch, tanpa URL, tanpa API route.

app/produk/page.tsx

```
import { getProduk } from '@/lib/produk'

export default async function Page() {
  const produk = await getProduk()

  return (
    <ul>
      {produk.map((p) => (
        <li key={p.id}>{p.title}</li>
      ))}
    </ul>
  )
}
```

Beda 1: HTML Awal Sudah Berisi Data

Hasil View Source halaman yang sama, dibangun dua cara. Ingat versi client dari video sebelumnya.

Client Component (useEffect)

```
<p>Memuat...</p>
```

Server Component

```
<ul>
  <li>Kopi</li>
  <li>Teh</li>
  <li>Roti</li>
</ul>
```

Ini jawaban untuk SEO

Daftar produk sudah ada di HTML yang pertama kali diterima, tanpa menunggu JavaScript jalan.

Beda 2: Mana State Loading dan Error?

Di komponen server tidak ada. Dua-duanya ditangani di luar komponen.

Komponen menunggu dulu, baru HTML dikirim

Saat saya uji dengan API yang sengaja lambat 0,8 detik, halaman baru tiba sekitar 0,8 detik kemudian, langsung lengkap.

Loading: loading.tsx atau Suspense

Menampilkan sesuatu selagi data diambil. Dibahas di sub-bab berikutnya.

Error: error.tsx

Kalau fetch melempar error, ditangani oleh error boundary, bukan state di komponen.

Memakai Secret di Server

API key dibaca dari environment variable, langsung di komponen.

app/produk/page.tsx

```
type Produk = { id: number; title: string }

export default async function Page() {
  // API_KEY hanya ada di server, tanpa NEXT_PUBLIC_
  const res = await fetch('https://api.contoh.com/produk', {
    headers: { Authorization: `Bearer ${process.env.API_KEY}` },
  })
  const produk: Produk[] = await res.json()

  return <p>{produk.length} produk</p>
}
```

Beda 3: Yang Tidak Ikut ke Browser

Saya cek isi file JavaScript hasil build dan HTML-nya.

Kode pengambil data

Pesan 'Gagal mengambil data' dari versi client ditemukan di JavaScript browser. Data dan kode versi server tidak ditemukan di sana.

Secret

Nilai API_KEY tidak ada di file JavaScript maupun di HTML. Yang sampai ke browser hanya hasil renderingnya.

Request-nya sendiri

Terjadi dari server ke server, jadi tidak muncul di tab Network dan tidak kena aturan CORS.

Beda 4: Kapan Datanya Diambil?

Perilaku default Next.js 16. Ini sering mengejutkan.

Static (tanda ○ di build)

Diambil sekali saat next build

- Datanya sama untuk semua pengunjung
- Sangat cepat (sekitar 40 ms di uji lokal)
- Data bisa basi sampai build berikutnya

Dynamic (tanda ƒ di build)

Diambil setiap ada request

- Datanya selalu terbaru
- Menunggu fetch selesai (sekitar 0,8 detik di uji lokal)
- Aktif kalau memakai API dinamis

Contoh di atas ternyata Static

Halaman yang hanya await fungsi data biasa otomatis diprerender saat build.

Membuat Halaman Dynamic

connection() memberi tahu Next.js: tunggu request asli sebelum merender.

app/produk/page.tsx

```
import { connection } from 'next/server'
import { getProduk } from '@/lib/produk'

export default async function Page() {
  await connection()
  const produk = await getProduk()

  return (
    <ul>
      {produk.map((p) => (
        <li key={p.id}>{p.title}</li>
      ))}
    </ul>
  )
}
```

Cache Components Mengubah Aturannya

Semua contoh di video ini memakai konfigurasi default Next.js.

Kalau cacheComponents: true diaktifkan

Halaman yang mengambil data tanpa dibungkus apa pun gagal saat build dengan pesan: Next.js encountered uncached or runtime data during prerendering.

Cara memperbaikinya

Bungkus dengan Suspense (video berikutnya) atau cache dengan 'use cache' (video berikutnya). Cache Components kita aktifkan di sub-bab caching.

Perbandingan Fetch di Client dan Server

Aspek	Client Component	Server Component
Cara menulis	useEffect + useState	async/await langsung
Request dikirim dari	Browser pengguna	Server
HTML awal	Belum berisi data	Sudah berisi data
Kode fetch ke browser	Ikut dikirim	Tidak ikut dikirim
Secret dan database	Tidak boleh	Aman dipakai
Loading dan error	State di komponen	loading.tsx, Suspense, error.tsx